

Задача А. Турнир в Зверополисе

Автор задачи и разработчик: Павел Скобелкин

Для начала разберём подгруппы, которые могут привести к полному решению.

Обратите внимание, что практически во всех подзадачах для корректной работы программы стоило использовать 64-битный тип данных — например, тип `long long` в C++.

В **первой подгруппе** $a = 0$, то есть Джуди не выиграла ни одной игры в турнире. Значит, удвоений у неё быть не могло — и за каждую ничью она получила 1 очко, за каждое поражение — 0 очков, то есть в таком случае минимальное и максимальное количество очков это b .

Сразу поймём, как считать **максимальное количество очков в произвольном случае**. Нам нужно выбрать, на результаты каких матчей тратить удвоения. Несложно понять, что если мы максимизируем очки, то все удвоения выгодно тратить на победные матчи. Таким образом, в оптимальном решении все победы должны стоять подряд. Далее несложно понять, что после серии побед выгоднее поставить ничью, чем победу (если у Джуди есть хотя бы одна ничья). Значит, максимальное количество баллов можно посчитать таким образом:

$$a \cdot 2 + b + 2 \cdot \max(a - 2, 0) + (a \geq 2 \ \&\& \ b > 0)$$

Далее будем в каждой подгруппе вычислять только минимальное количество баллов.

Не будем подробно разбирать подгруппы 2–7 — в основном, их можно было пройти, придумав/разобрав часть случаев из полного решения.

Подзадачу 8 можно было решить, перебрав все возможные комбинации побед, ничьих и проигрышей. С помощью рекурсивного перебора можно было добиться асимптотики $\mathcal{O}(3^n n)$, где $n = a + b + c$, чего достаточно для решения подзадачи.

Подзадача 9 решалась с помощью динамического программирования. Не будем углубляться в детали, но в качестве состояния ДП можно было взять: кол-во оставшихся побед, кол-во оставшихся ничьих, кол-во оставшихся проигрышей, и текущая серия побед (0, 1 или ≥ 2). Таким образом, было порядка n^3 состояний ДП, каждое из которых можно было пересчитать за $\mathcal{O}(1)$, добившись асимптотики $\mathcal{O}(n^3)$.

Обратим внимание, что с помощью подзадач 8 и 9 можно было догадаться до формулы для полного решения — с помощью анализа ответов на «маленькие» тесты.

Полное решение.

При минимизации мы хотим, чтобы удвоения как можно чаще «сгорали» на поражениях (дают 0 даже после удвоения) и как можно реже попадали на игры с положительными очками (победы и ничьи). Основную роль играют **блоки побед**: последовательности подряд идущих W , разделённые не-победами.

- Если в блоке побед длина ≥ 3 , то начиная с третьей победы внутри блока победы неизбежно удваиваются.
- После каждого фрагмента WW (двух побед подряд) следующий матч (если он существует) будет удвоен.

Всего не-побед $n = b + c$, значит блоков побед не больше $n + 1$.

Случай 1: $a \leq n + 2$

Покажем, что можно добиться полного отсутствия полезных удвоений (то есть ни одна победа и ни одна ничья не удвоится).

Разобьём победы по блокам так, чтобы почти все блоки имели длину 1, и (если нужно) ровно один блок имел длину 2.

Блок длины 2 поставим в самый конец, тогда после него нет матча, и удвоение не сработает вообще. Следовательно, сумма очков равна просто базовой:

$\text{min} = 2a + b$

Случай 2: $n + 2 < a \leq 2n + 2$

Здесь всё ещё можно избежать блоков длины ≥ 3 , то есть сделать все блоки побед длины 1 или 2, но блоков длины 2 уже несколько.

Пусть t — число блоков длины 2. Если сначала положить по одной победе в каждый из $n + 1$ блоков, получим $n + 1$ побед, а оставшиеся $a - (n + 1)$ побед будут «добавлять вторую победу» в некоторых блоках. Поэтому

$$t = a - (n + 1) = a - (b + c + 1).$$

Так как $a > n + 2$, имеем $t \geq 2$.

Один из блоков WW снова поставим в самый конец, тогда после него удвоения нет. Остается $t - 1$ блоков WW , после каждого из которых есть следующий матч, и он удваивается.

Чтобы минимизировать очки, мы хотим, чтобы эти $t - 1$ удвоений пришлись на поражения. Поражений всего c , значит как минимум

$$\max(0, (t - 1) - c)$$

раз удвоение будет вынужденно попадать на ничью, что увеличивает сумму на $+1$ за каждую такую ситуацию.

Подставим $t = a - (b + c + 1)$:

$$(t - 1) - c = a - (b + c + 1) - 1 - c = a - b - 2c - 2.$$

В таком случае,

$\min = 2a + b + \max(0, a - b - 2c - 2)$

Случай 3: $a > 2n + 2$

Если все блоки побед имеют длину ≤ 2 , то побед максимум $2(n + 1) = 2n + 2$. При $a > 2n + 2$ блок длины ≥ 3 неизбежен, то есть появятся удвоенные победы.

Оптимальная (для минимума) конструкция: сделаем ровно $n + 1$ блоков побед и перед каждой не-победой поставим WW , то есть

$$(WWX_1)(WWX_2)\dots(WWX_n) \text{ и все оставшиеся победы в конец,}$$

где X_i — все b ничьих и c поражений в каком угодно порядке. Тогда:

- каждая из n не-побед стоит после WW , значит она удвоена;
- внутри каждого блока победы с 3-й и далее удвоены (это уже неизбежно при таком a).

Посчитаем очки. Рассмотрим сумму по победам. В каждом блоке длины $L \geq 2$: первые две победы дают $2 + 2 = 4$, а остальные $L - 2$ побед дают по 4 каждая, итого $4L - 4$. Если блоков $n + 1$, а сумма длин блоков равна a , то

$$\text{очки за победы} = \sum (4L_i - 4) = 4a - 4(n + 1) = 4a - 4(b + c) - 4.$$

Теперь не-победы. Каждая ничья удвоена и даёт 2 (то есть суммарно $2b$), поражения дают 0. Итого

$$\min = (4a - 4(b + c) - 4) + 2b = 4a - 2b - 4c - 4.$$

Значит

$\min = 4a - 2b - 4c - 4$

Доказательство оптимальности этих жадных конструкций остается на упражнение читателю — жюри находит их достаточно интуитивными.

Итоговые формулы

$$\max = 2a + b + 2 \max(a - 2, 0) + [a \geq 2 \wedge b > 0]$$

$$\min = \begin{cases} 2a + b, & a \leq b + c + 2, \\ 2a + b + \max(0, a - b - 2c - 2), & b + c + 2 < a \leq 2(b + c) + 2, \\ 4a - 2b - 4c - 4, & a > 2(b + c) + 2. \end{cases}$$

Таким образом, для решения задачи нужно было проверить выполнение одного из трех условий и вывести формулу — итоговая асимптотика равна $\mathcal{O}(1)$.

Задача В. Тренировка по пейнтболу

Автор задачи и разработчик: Олег Сметанкин

Разберем каждую подзадачу, чтобы прийти к итоговому решению.

Первая подзадача сводилась к перебору t_0 и полной симуляции движения щита вверх с проверкой всех шаров.

Во **второй подзадаче** можно было перебрать t_0 и для i -го шарика вывести формулу положения щита:

$$d = t_i - t_0$$

Тогда при выполнении условий:

$$0 \leq d \leq h - w, \quad 1 + d \leq s_i \leq w + d$$

шарик будет заблокирован.

В **третьей подзадаче** щит существует ровно одну секунду, то есть в t_0 , и блокирует все поле, поэтому подзадача сводится к поиску максимального количества шариков, которые летят одновременно. Это может быть реализовано словарем (map).

В **четвертой подзадаче** заметим, что в секунду $t_0 + d$ щит будет закрывать ряд $1 + d$. Тогда легко вывести формулу:

$$s_i = 1 + (t_i - t_0) \iff t_0 = t_i - (s_i - 1).$$

То есть каждый шарик однозначно определяет t_0 , который его заблокирует. Чтобы получить ответ, найдем максимальное количество одинаковых t_0 среди всех шаров с помощью словаря.

Все следующие подзадачи требуют математически вывести отрезок подходящих t_0 . В секунду t_i щит (если существует) находится на сдвиге $d = t_i - t_0$ и закрывает ряды

$$[1 + d, w + d] = [1 + (t_i - t_0), w + (t_i - t_0)].$$

Тогда шарик попадет в щит:

$$1 + (t_i - t_0) \leq s_i \leq w + (t_i - t_0).$$

Переносим t_0 :

$$t_i - (s_i - 1) \leq t_0 \leq t_i + (w - s_i).$$

Также щит должен существовать в секунду t_i :

$$t_0 \leq t_i \leq t_0 + (h - w) \iff t_i - (h - w) \leq t_0 \leq t_i.$$

Пересекаем два отрезка по t_0 и получаем итоговый, не забывая, что t_0 должен быть целым и положительным:

$$l_i = \max(t_i - (s_i - 1), t_i - (h - w), 1), \quad r_i = \min(t_i + (w - s_i), t_i).$$

Если $l_i \leq r_i$, то для любого целого t_0 из $[l_i, r_i]$ пуля будет заблокирована. После вывода этих границ, задача сводится к поиску точки, покрытой наибольшим числом отрезков, то есть к стандартной задаче на сканирующую прямую (сканлайн), с которым получалось полное решение.

В 5 подзадаче предполагался правильный вывод границ и медленный поиск нужной точки за $O(n^2)$ с перебором кандидатов и отрезков.

В 6 подзадаче после вывода границ вместо сканлайна можно было использовать разностный массив.

Задача С. Погоня за патентом

Автор задачи и разработчик: Антон Вдовин

1 Подгруппа 1

Для первой подгруппы ограничения маленькие: $n, q \leq 50$, $a_i \leq 100$. Поэтому можно решать «в лоб». Для каждого запроса $[l, r]$ перебираем d от 2 до 100 и проверяем, что для всех $i \in [l, r]$ выполнено $\gcd(d, a_i) = 1$. Первое подходящее d — ответ.

Сложность: на один запрос мы перебираем до A значений d и для каждого пробегаем отрезок длины до n , итого $O(q \cdot n \cdot A)$, где $A = \max(a_i)$ (в этой подгруппе $A \leq 100$).

2 Подгруппа 2

Здесь уже нельзя перебирать все d «до бесконечности», но всё ещё можно перебирать разумное количество кандидатов. Ключевое наблюдение: оптимальное d всегда можно выбрать простым.

Доказательство: пусть некоторое составное d подходит, то есть $\gcd(d, a_i) = 1$ для всех i на отрезке. Тогда любой простой делитель $p \mid d$ тоже будет удовлетворять $\gcd(p, a_i) = 1$ для всех i : иначе p делил бы какой-то a_i , а значит делил бы и d , что противоречит взаимной простоте. Причём $p < d$, значит минимальный ответ обязан быть простым.

В подгруппе 2 достаточно перебрать все простые p от 2 до A (в ограничениях этой подгруппы $A \leq 500$, простых там около 95) и для каждого запроса проверить, что на отрезке нет чисел, делящихся на p .

Сложность: $O(q \cdot n \cdot \pi(A))$, где $\pi(A)$ — количество простых $\leq A$.

3 Подгруппа 3

В этой подгруппе n, q уже большие, но a_i очень «простые» (например, $a_i \leq 3$, то есть значения только из $\{1, 2, 3\}$). Число 1 не влияет на ответ, так как $\gcd(d, 1) = 1$ при любом d .

На отрезке возможны три случая:

1. если на отрезке нет числа 2, то ответ $d = 2$;
2. иначе, если на отрезке нет числа 3, то ответ $d = 3$;
3. иначе (есть и 2, и 3), то минимальный простой, отсутствующий среди делителей на отрезке, равен 5, и ответ $d = 5$.

Чтобы отвечать быстро, заведём префиксные суммы:

$$cnt2[i] = \#\{j \leq i : a_j = 2\}$$

$$cnt3[i] = \#\{j \leq i : a_j = 3\}.$$

Тогда число двоек на отрезке $[l, r]$ равно $cnt2[r] - cnt2[l - 1]$, аналогично для троек. По ним мгновенно выбираем ответ.

Важно заметить идею, которая пригодится дальше: мы фактически ищем «минимальное простое, отсутствующее среди простых делителей чисел на отрезке», то есть это похоже на *MEX* по простым.

4 Подгруппа 4

В этой подгруппе каждый запрос — это один элемент, то есть $l = r$. Нужно для каждого числа $x = a_l$ найти минимальное $d \geq 2$, такое что $\gcd(d, x) = 1$.

Мы уже знаем, что оптимальный ответ можно выбрать простым, значит задача сводится к поиску минимального простого p , которое не делит x . Пусть $S(x)$ — множество различных простых делителей числа x (без кратностей). Тогда ответ — это минимальное простое p , которого нет в $S(x)$.

Решение: разложить x на простые множители перебором делителей до $\sqrt{x}$, собрать множество $S(x)$, после чего проверить простые $2, 3, 5, 7, \dots$ по возрастанию, пока не найдём первый, который не делит x .

Сложность: $\mathcal{O}(\sqrt{x})$ на факторизацию каждого запроса.

5 Подгруппа 5

Как и в подгруппе 4, запросы точечные ($l = r$), но ограничения больше, и факторизовать за $\mathcal{O}(\sqrt{x})$ уже невыгодно. Нужно ускорить нахождение $S(x)$.

Делаем решето и считаем spf (минимальный простой делитель) до $A = \max(a_i)$. Для каждого числа x можно получать его разложение, постоянно деля на $spf[x]$, выкидывая повторения. Тогда множество $S(x)$ находится примерно за $\mathcal{O}(\log x)$ шагов деления.

После нахождения $S(x)$ снова проверяем простые $2, 3, 5, 7, \dots$ пока не найдём минимальное, отсутствующее в $S(x)$.

Сложность: построение spf за $\mathcal{O}(A)$, ответы на запросы суммарно $\mathcal{O}(q \cdot \log A)$.

6 Подгруппа 6

Здесь можно опереться на общую формулу ответа. Пусть на отрезке стоят числа $a_l, a_{l+1}, \dots, a_r$. Обозначим через $S(x)$ множество простых делителей числа x . Тогда простое p запрещено как ответ тогда и только тогда, когда оно делит хотя бы один элемент отрезка, то есть $p \in \bigcup_{i=l}^r S(a_i)$.

Следовательно,

$$answer(l, r) = MEX\left(\bigcup_{i=l}^r S(a_i)\right),$$

где MEX берётся по простым числам (то есть минимальное простое, не входящее в объединение).

В подгруппе 6 условие упрощает факторизацию: например, если все a_i — простые, то $S(a_i)$ содержит ровно один элемент, и при добавлении/удалении позиции мы обновляем ровно один простой.

Тогда удобно применить алгоритм МО:

- сортируем запросы по блокам (по l) и по r ;
- поддерживаем текущий отрезок и частоты $cnt[p]$ — сколько раз простой p встретился среди делителей текущего отрезка;
- при сдвиге границ добавляем/удаляем простые из $S(a_i)$.

Чтобы быстро находить MEX по простым, можно:

- либо поддерживать указатель на текущий ответ и двигать его вправо, пока частота не станет 0 (иногда это работает амортизированно),
- либо хранить частоты по индексам простых и сделать блочную декомпозицию, чтобы быстро находить первый индекс с нулевой частотой.

7 Подгруппа 7 – 9

Теперь ограничения уже близки к полным, поэтому нужно:

1. быстро получать $S(a_i)$ для каждого a_i ;
2. быстро пересчитывать MEX при изменении отрезка.

Первый пункт решаем так же, как в подгруппе 5: строим spf до $A = \max(a_i)$ и заранее факторизуем каждое a_i , сохраняя список различных простых делителей (без повторов). Для $a_i \leq 10^7$ число различных простых делителей маленькое, поэтому обновления при добавлении/удалении элемента быстрые.

Дальше для обработки запросов всё ещё подходит алгоритм МО:

- оффлайн сортировка запросов;
- поддерживаем частоты по простым $cnt[p]$;
- при $add/remove$ обновляем все $p \in S(a_i)$.

Поиск MEX по простым делаем через блочную декомпозицию по индексам простых:

- нумеруем простые $primes[0..P-1]$;
- храним частоты по индексам и для каждого блока число нулей;
- MEX ищем так: первый блок, где есть ноль, затем первый ноль внутри блока.

8 Подгруппа 10

В полной подгруппе хочется уйти от МО и сделать решение почти $\mathcal{O}((n+q) \cdot \log P)$.

Для простого p условие « p подходит для запроса $[l, r]$ » эквивалентно тому, что на отрезке нет элемента, делящегося на p .

Определим для каждого простого p значение $last[p]$ — последняя позиция, на которой встречался элемент, кратный p . Тогда для фиксированного r и запроса $[l, r]$ простой p отсутствует на отрезке тогда и только тогда, когда $last[p] < l$. А значит ответ — это минимальный простой p , для которого $last[p] < l$.

Как это реализовать:

1. Строим spf и список простых $primes[]$ до $A = \max(a_i)$.
2. Идём $r = 1..n$: для текущего a_r находим список различных простых делителей $S(a_r)$ и для каждого $p \in S(a_r)$ делаем $last[p] = r$.
3. Все запросы группируем по правой границе: обрабатываем запросы, когда дошли до их r .
4. Строим Дерево Отрезков по индексам простых (по массиву $primes[]$), где в вершине хранится минимум $last$ на соответствующем диапазоне простых.

Обновление: точно обновляем индекс простого p значением $last[p] = r$.

Запрос: нужно найти самый левый индекс j , где минимум на каком-то префиксе стал $< l$. Это делается спуском по дереву: если минимум в левом сыне $< l$, идём влево, иначе идём вправо.

Итого:

- на каждый r делаем до нескольких обновлений (по числу различных простых делителей a_r);
- каждое обновление $\mathcal{O}(\log P)$;
- каждый запрос $\mathcal{O}(\log P)$.

Общая сложность порядка $\mathcal{O}((n+q) \cdot \log P)$ с небольшим множителем на факторизацию через spf .

Задача D. Рабочие пары в Зверополисе

Автор задачи и разработчик: Дмитрий Грунтов

9 $n \leq 2\,000$

Для каждого зверя рассмотрим всех других зверей и посчитаем эффективность такой пары за $\mathcal{O}(\log(n))$, переводя числа в двоичную систему счисления. Асимптотика такого решения $\mathcal{O}(n^2 \log(n))$.

10 $n \leq 10\,000$

Оптимизируем прошлое решение с помощью встроенных в C++ битовых операций, чтобы избавиться от логарифма в асимптотике. Перебирать биты вручную больше не нужно: используем то, что функция $F(x, y)$ зависит только от общих единичных битов.

Пусть $z = x \& y$. Тогда $F(x, y)$ — это количество позиций i , где в z стоят единицы и на i , и на $i + 1$, то есть количество единиц в числе $z \& (z \gg 1)$:

$$F(x, y) = \text{popcount}((x \& y) \& ((x \& y) \gg 1)).$$

Количество единичных бит можно посчитать за $\mathcal{O}(1)$ стандартной функцией `__builtin_popcount`.

Итоговая асимптотика решения $\mathcal{O}(n^2)$.

11 $n \leq 100\,000$

Сразу сделаем описанное в прошлой подгруппе преобразование над эффективностями зверей: $b_i = a_i \& (a_i \ll 1)$. После такого преобразования у нас останется $G(n)$ различных чисел. Для $n = 100\,000$ получаем $G(n) = 6079$ (то есть примерно $6 \cdot 10^3$) различных масок, поэтому можно перейти от перебора всех n^2 пар к перебору только различных значений. Таким образом, получаем решение за $\mathcal{O}(G^2(n))$.

12 $n \leq 200\,000$

Построим ДП по маскам. Пусть dp_{mask} — количество зверей, у которых есть $mask$ как подмаска.

Чтобы получить ответ для зверя с эффективностью x , нужно найти такую подмаску его текущей маски (то есть, $x \& mask = mask$), имеющую наибольшее количество единичных битов и $dp_{mask} \geq 2$.

Таким образом, решение может выглядеть так: переберем все возможные маски и их подмаски; Для текущей маски посчитаем ответ — максимальное количество единичных бит в подмаске с $dp_{mask} \geq 2$. Воспользовавшись техникой перебора всех масок и всех их подмасок за 3^{len} , получим решение за $3^{\log_2 n} = n^{\log_2 3} \approx \mathcal{O}(n^{1.585})$.

13 Полное решение

Используем SOS DP, чтобы не перебирать все подмаски явно. Пусть $cnt[mask]$ — сколько раз встречается маска $mask$.

Зададим базовые значения:

$$dp_2[mask] = \begin{cases} \text{popcount}(mask), & cnt[mask] \geq 2, \\ -\infty, & \text{иначе.} \end{cases}$$

Далее протолкнём максимум с подмасок на надмаски. Для $k = 0 \dots B - 1$, для всех $mask$:

$$\text{если } (mask \& (1 \ll k)) \neq 0, \quad dp_2[mask] \leftarrow \max(dp_2[mask], dp_2[mask \oplus (1 \ll k)]),$$

то есть мы рассматриваем $x = mask \oplus (1 \ll k)$, полученный удалением одного единичного бита. После этого

$$dp_2[mask] = \max_{s \subseteq mask} dp_2[s]$$

Асимптотика: $\mathcal{O}(B \cdot 2^B + n)$, где $B = \log_2 n + 1$.